# Financial Assets Platform

## Project Overview

Financial Assets Platform is a Java Spring Boot and MongoDB based platform for ingesting, storing, versioning, querying, analyzing, and explaining financial market data.

The system was designed to demonstrate:

- NoSQL data storage
- Temporal data management
- Provenance tracking
- ETL-style ingestion pipelines
- REST API development
- Analytics and forecasting
- Assistant tools and agent workflow
- Local LLM integration
- Unit and repository testing

The platform ingests market data from external providers such as Alpha Vantage and Stooq, stores it in MongoDB, preserves full historical versions, performs analytics, and exposes both API and assistant-style interfaces.

---

# Architecture

The application follows a layered architecture.

## Controller Layer

Responsible for exposing REST endpoints.

Controllers:

- AssetController
- DataProviderController
- MarketDataController
- IngestController
- AnalyticsController
- AssistantController

## Service Layer

Contains business logic.

Services:

- IngestService
- OllamaService

## Repository Layer

Responsible for MongoDB access.

Repositories:

- AssetRepository
- DataProviderRepository
- MarketDataRepository
- ProviderMetadataRepository

## Database

MongoDB is used as the primary NoSQL database.

The project uses Spring Data MongoDB repositories and document-based storage.

---

# Core Domain Model

## Asset

Represents a financial instrument.

Examples:

- IBM
- AAPL
- MSFT

Stored information:

- assetId
- ticker
- assetName
- category
- attributes

---

## DataProvider

Represents an external financial data source.

Examples:

- ALPHA_VANTAGE
- STOOQ

Stored information:

- providerId
- providerName
- baseUrl
- description
- attributes

---

## MarketData

Represents a financial time-series observation.

Stored information:

- assetId
- providerId
- dataDate
- openPrice
- highPrice
- lowPrice
- closePrice
- volume

Additional provenance information:

- sourceEndpoint
- rawProviderSymbol
- rawDataHash
- ingestionTime

---

# NoSQL Storage

MongoDB was selected because:

- financial market data is naturally document-oriented
- schema flexibility is required
- temporal versioning generates many document versions
- heterogeneous metadata is supported through dynamic attribute maps

The application uses:

```
@Document
```

annotations and Spring Data MongoDB repositories.

---

# Temporal Data Management

A key requirement of the project is temporal support.

The platform never performs destructive updates.

Instead:

1. Current record is closed:

```
current = false
validUntil = timestamp
```

1. New version is inserted:

```
current = true
validFrom = timestamp
```

1. Logical deletion is implemented through:

```
deleted = true
```

Historical versions remain available indefinitely.

---

# Temporal Features Demonstrated

The demo includes:

## Temporal Asset Update

Creates a new version of an asset.

## Asset History

Retrieves all historical versions of an asset.

### Temporal Market Data Update

Creates a new version of a market data record.

### Market Row History

Retrieves all versions of a market data observation.

---

# Provenance Tracking

Every ingested market record stores:

- source endpoint
- ingestion timestamp
- provider symbol
- content hash

This allows:

- auditability
- reproducibility
- lineage tracking

---

# ETL Ingestion

The ingestion workflow follows ETL principles.

### Extract

Data is downloaded from:

- Alpha Vantage
- Stooq

### Transform

CSV data is:

- parsed
- normalized
- validated
- converted into MarketData objects

### Load

Data is persisted into MongoDB.

---

# Supported Providers

## Alpha Vantage

Endpoint:

```
POST /api/ingest/alpha-vantage
```

Example:

```
POST /api/ingest/alpha-vantage?symbol=IBM&assetId=IBM
```

---

## Stooq

Endpoint:

```
POST /api/ingest/stooq
```

Example:

```
POST /api/ingest/stooq?symbol=ibm&assetId=IBM
```

---

# Idempotent Ingestion

The ingestion process is safe to rerun.

Behavior:

- unchanged rows are skipped
- modified rows create new temporal versions
- duplicate records are avoided

This prevents accidental data duplication.

---

# REST API

## Assets

```
GET    /api/assets
GET    /api/assets/{assetId}
POST   /api/assets
PUT    /api/assets/{assetId}
DELETE /api/assets/{assetId}
```

---

## Providers

```
GET    /api/providers
GET    /api/providers/{providerId}
POST   /api/providers
PUT    /api/providers/{providerId}
DELETE /api/providers/{providerId}
```

---

## Market Data

```
GET    /api/market-data
GET    /api/market-data/search
GET    /api/market-data/{timeSeriesId}
POST   /api/market-data
PUT    /api/market-data/{timeSeriesId}
DELETE /api/market-data/{timeSeriesId}
```

---

# Pagination

To improve scalability, list endpoints support pagination.

Supported parameters:

```
offset
limit
```

Examples:

```
GET /api/assets?offset=0&limit=10

GET /api/providers?offset=0&limit=10

GET /api/market-data?offset=0&limit=50
```

This prevents returning excessively large result sets.

---

# Time-Series Queries

The platform supports:

## Current Data

Retrieve latest active records.

## Historical Versions

Retrieve complete version history.

## Date Range Queries

Example:

```
GET /api/market-data/search
```

Parameters:

```
assetId
providerId
startDate
endDate
```

---

# Analytics Module

The analytics module computes financial statistics from stored market data.

---

## Analytics Summary

Computes:

- minimum close price
- maximum close price
- average close price
- trend direction
- percentage change
- absolute change

Endpoint:

```
GET /api/analytics/summary
```

---

## Forecast Next Day

Generates a next-day prediction using recent historical data.

Endpoint:

```
GET /api/analytics/forecast
```

---

## Compare Assets

Compares asset performance.

Endpoint:

```
GET /api/analytics/compare
```

---

## Export for ML/Spark

Exports processed market data for analytics workflows.

Endpoint:

```
GET /api/analytics/export
```

The export endpoint was designed to provide structured data that can later be consumed by analytics pipelines and Spark-based workflows.

---

# Assistant Layer

The platform includes an assistant layer exposing analytical tools.

Available tools:

- list_assets
- get_asset_details
- list_data_sources
- fetch_time_series
- summarize_trends
- forecast_next_day
- compare_assets
- explain_change
- agent_demo
- llm_chat

---

# Agent Workflow

The project contains an agent demonstration endpoint.

The agent can:

1. Inspect available tools
2. Select an appropriate tool
3. Retrieve platform information
4. Produce an explanation

---

# Local LLM Integration

The project integrates a real local Large Language Model using Ollama.

Model:

```
llama3.2
```

Communication is performed through:

```
http://localhost:11434/api/generate
```

using the OllamaService component.

---

# Grounded LLM Responses

Endpoint:

```
POST /api/assistant/llm-chat
```

Example:

```
{
  "message": "Explain the trend for IBM",
  "assetId": "IBM",
  "providerId": "ALPHA_VANTAGE"
}
```

Workflow:

1. Analytics data is retrieved from MongoDB.
2. Context is prepared.
3. Context is sent to Ollama.
4. The model generates an answer.
5. The response includes grounding information.

This reduces hallucinations and ensures responses are based on platform data.

---

# Testing

The project contains automated tests covering multiple layers.

Implemented tests:

- FinancialAssetsApplicationTests
- TemporalModelTest
- AssetControllerTest
- AnalyticsControllerTest
- MarketDataRepositoryTest
- IngestServiceTest

Validated functionality:

- application startup

- temporal versioning
- repository queries
- analytics calculations
- ingestion workflows
- controller behavior

---

# MongoDB Query Strategy

Current-version retrieval:

```
findByCurrentTrueAndDeletedFalse()
```

Historical retrieval:

```
findByTimeSeriesIdOrderByValidFromDesc()
```

Temporal retrieval:

```
findValidMarketDataAt(...)
```

---

# Recommended Index Strategy

Recommended MongoDB indexes:

```
assetId
providerId
dataDate
current
deleted
validFrom
validUntil
```

These indexes optimize:

- temporal queries
- historical lookups
- analytics filtering
- current-version retrieval

---

# Technologies Used

- Java 21
- Spring Boot 3
- MongoDB
- Spring Data MongoDB
- Swagger / OpenAPI
- JUnit 5
- Mockito
- Ollama
- Llama 3.2

---

# Running the Application

## Start MongoDB

Local installation or Docker:

```
docker run -p 27017:27017 mongo
```

---

## Start Ollama

```
ollama run llama3.2
```

---

## Start Spring Boot

Run:

```
FinancialAssetsApplication
```

from IntelliJ IDEA.

---

# Demonstration Workflow

Recommended demo order:

1. Ingest Alpha Vantage
2. List Assets

3. Asset Details
4. List Providers
5. Provider Details
6. Time Series + Chart
7. Time Series As Of
8. Analytics Summary
9. Forecast Next Day
10. Export for ML/Spark
11. Assistant Tools
12. Assistant Summarize Trends
13. Assistant Forecast
14. Assistant Explain Change
15. Agent Demo
16. Temporal Update Asset
17. Asset History
18. Temporal Update Latest Market Row
19. Market Row History
20. Assistant Ask Chat (Ollama)

This workflow demonstrates ingestion, storage, temporal versioning, analytics, assistant tooling, local LLM integration, and historical data management end-to-end.